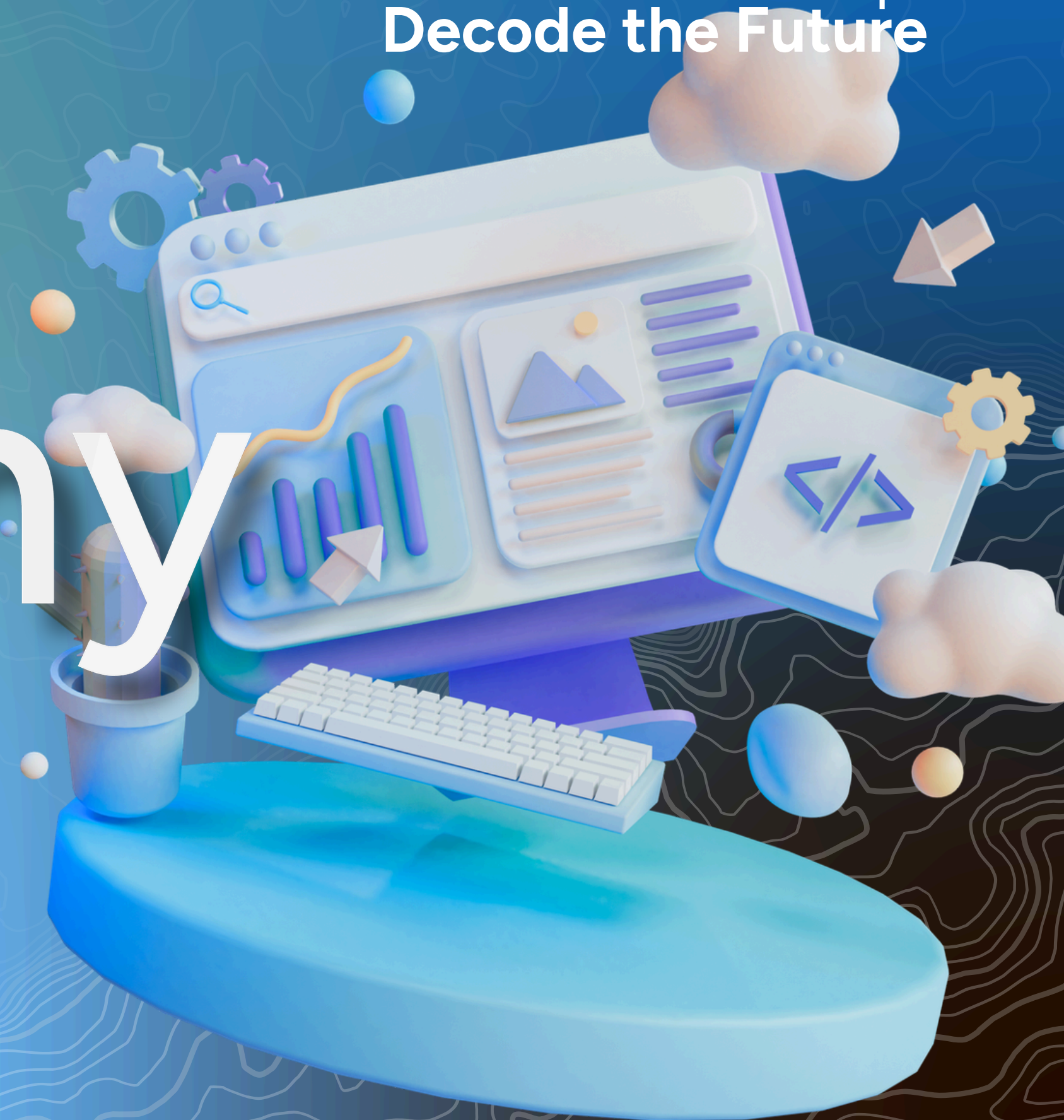


2026

Code the Map
Decode the Future

MAPID Academy

WebGIS Development Bootcamp



Learning Journey

WebGIS Course - Modern JavaScript

Focused :

1. Arrow Function, Destructuring & Spread/Rest
2. Optional Chaining, Array Methods (map/filter/reduce)
3. Modules & Async/Await

MAPID Academy

WebGIS Development Bootcamp

Code the Map
Decode the Future

02

Code the Map
Decode the Future

MODERN JAVASCRIPT

Apa itu Modern JavaScript

Code the Map
Decode the Future

Modern JavaScript mengacu pada fitur-fitur yang diperkenalkan sejak ES6 (ECMAScript 2015) dan versi setelahnya. Fitur ini membuat kode lebih singkat, mudah dibaca, dan aman.

ECMAScript (ES) adalah standar resmi bahasa JavaScript. ES6 (2015) adalah update terbesar yang mengubah cara kita menulis JS. Versi setelahnya (ES2016, ES2017, ...) menambahkan fitur baru setiap tahun.

Kenapa Belajar Modern JavaScript?

Code the Map
Decode the Future

Manfaat	Contoh
Kode lebih singkat	Arrow function, destructuring
Lebih mudah dibaca	Template literal, async/await
Lebih aman dari error	Optional chaining, nullish coalescing
Standar industri	Hampir semua kode & library modern memakainya
Wajib untuk framework	React, Vue, Node.js semua pakai ES6+

let & const

Code the Map
Decode the Future

Di JavaScript lama, hanya ada var. ES6 memperkenalkan let dan const yang lebih aman karena memiliki block scope.

Aspek	var (lama)	let / const (modern)
Scope	Function scope	Block scope { }
Re-deklarasi	Boleh (rawan bug)	Tidak boleh
Hoisting	(jadi undefined)	(error jika dipakai duluan)
Rekomendasi	Hindari	Selalu pakai ini

Arrow Function

Code the Map
Decode the Future

Arrow function adalah cara singkat menulis function. Sangat sering dipakai, terutama di dalam array methods dan callback.

Kapan Pakai Arrow Function?

Function biasa

```
arr.forEach(function(x) {  
  console.log(x);  
});
```

Arrow (lebih ringkas)

```
arr.forEach(x => {  
  console.log(x);  
});
```

Destructuring

Code the Map
Decode the Future

Destructuring adalah cara cepat "membongkar" isi array atau object ke dalam variabel. Sangat berguna untuk data GeoJSON!

Array Destructuring

```
// Cara lama
const koordinat = [107.61, -6.90];
const lng = koordinat[0];
const lat = koordinat[1];

// Cara modern — destructuring (urutan sesuai posisi)
const [lng, lat] = [107.61, -6.90];
console.log(lng); // 107.61
console.log(lat); // -6.90

// Contoh nyata di GeoJSON:
const feature = { geometry: { coordinates: [107.68, -6.90] } };
const [bujur, lintang] = feature.geometry.coordinates;
```

Object Destructuring

```
const lokasi = {
  nama: "Apartemen Transit",
  kota: "Bandung",
  status: "BUKA"
};

// Cara lama
const nama = lokasi.nama;
const kota = lokasi.kota;

// Cara modern — destructuring (nama variabel = nama key)
const { nama, kota, status } = lokasi;
console.log(nama); // "Apartemen Transit"
console.log(kota); // "Bandung"

// Ganti nama variabel dengan :
const { nama: namaLokasi } = lokasi;
console.log(namaLokasi); // "Apartemen Transit"

// Nilai default jika key tidak ada
const { telepon = "-" } = lokasi;
console.log(telepon); // "-" (karena lokasi tidak punya telepon)
```


Array Methods Modern

Code the Map
Decode the Future

nti dari mengolah data di JavaScript modern. Semua menerima callback (biasanya arrow function) dan tidak mengubah array asli (kecuali sort).

Method	Fungsi	Mengembalikan
map()	Ubah setiap item	Array baru (ukuran sama)
filter()	Saring item sesuai kondisi	Array baru (lebih kecil)
find()	Cari 1 item pertama yang cocok	1 item (atau undefined)
reduce()	Gabungkan jadi 1 nilai	1 nilai (angka/object/dll)
some()	Cek apakah ADA yang cocok	true / false
every()	Cek apakah SEMUA cocok	true / false
sort()	Urutkan array	Array terurut (ubah asli!)

Modules (import / export)

Code the Map
Decode the Future

Module memungkinkan kita memecah kode menjadi file-file terpisah yang saling terhubung. Membuat kode lebih rapi dan mudah dikelola.

Named Export

```
// === file: utils.js ===  
export const PI = 3.14;  
  
export function hitungJarak(a, b) {  
  return Math.abs(a - b);  
}  
  
// === file: main.js ===  
import { PI, hitungJarak } from "./utils.js";  
  
console.log(PI); // 3.14  
console.log(hitungJarak(10, 3)); // 7
```

Default Export

```
// === file: peta.js ===  
export default function buatPeta() {  
  return "Peta dibuat!";  
}  
  
// === file: main.js ===  
// default export — nama saat import bebas, tanpa { }  
import buatPeta from "./peta.js";  
  
console.log(buatPeta()); // "Peta dibuat!"
```


Promise & async/await

Code the Map
Decode the Future

Di Sesi 8 kita sudah pakai async/await untuk fetch. Berikut pemahaman lebih dalam tentang Promise yang ada di baliknya.

Apa itu **Promise**?

Promise adalah "janji" hasil dari operasi asynchronous. Punya 3 status: pending (menunggu), fulfilled (berhasil), rejected (gagal).

```
// fetch() mengembalikan sebuah Promise
const promise = fetch(url);
console.log(promise); // Promise { <pending> }

// .then() = saat berhasil, .catch() = saat gagal
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Promise & async/await

Code the Map
Decode the Future

Di Sesi 8 kita sudah pakai async/await untuk fetch. Berikut pemahaman lebih dalam tentang Promise yang ada di baliknya.

async/await (Cara Modern)

```
async function ambilLokasi() {
  try {
    const response = await fetch(url);
    const data = await response.json();

    // destructuring + optional chaining + array methods
    (gabungan ES6+!)
    const nama = data?.features
      ?.filter(f => f.properties?.NAMA)
      .map(f => f.properties.NAMA);

    console.log(nama);
  } catch (error) {
    console.error("Gagal:", error);
  }
}
```

Promise.all — Beberapa Request Sekaligus

```
// Jalankan beberapa fetch BERSAMAAN (lebih cepat!)
async function ambilSemua() {
  const [lokasi, cuaca] = await Promise.all([
    fetch(urlLokasi).then(r => r.json()),
    fetch(urlCuaca).then(r => r.json())
  ]);

  console.log(lokasi, cuaca); // kedua data sudah siap
}

// Tanpa Promise.all: harus menunggu satu per satu (lebih lambat)
```


Task...

Code the Map
Decode the Future

Dari hasil penugasan sebelumnya (membuat halaman web dengan HTML dan CSS), silahkan terapkan Javascript untuk membuat web menjadi lebih interaktif.

Poin-poin yang harus ada:

1. Interaksi dinamis user pada halaman web (misal elemen berubah warna saat user mengklik suatu button, elemen muncul/hilang ketika user mengklik suatu button dll).
2. Menampilkan Data dinamis yang didapatkan melalui hasil fetch dari API Geomapid (misal memunculkan card berisi informasi lengkap setiap apartemen di geomapid)